Ca' Foscari University of Venice

Department of Environmental Sciences, Informatics and Statistics

Course Title : SOFTWARE ARCHITECTURES
Course Code: CM0639-1
Assignment: Task 1 & Task 2

Prepared by:
Student Name: Yared Debela
Student Id: 913882
Instructor : Prof. Pietro Ferrara
Academic Year: 2025-2026

Submission Date: 02-01-2026

Task 1 – System Architecture Design

Team-Based Soccer Tournament Management System

1. Introduction

This project focuses on the design and implementation of an IT system for managing team-based soccer tournaments. The system supports the complete lifecycle of a tournament, including team and player registration, match pairing and scheduling, match event tracking, result recording, standings calculation, and public dissemination of tournament information.

The system is fully implemented using a distributed microservices architecture. Each core domain concern is developed as an independent service with its own database, REST APIs, and event-driven communication. This architectural decision was made to satisfy scalability, performance, reliability, deployability, maintainability, and cost-efficiency requirements defined for the tournament system.

2. Scope and Domain Definition

2.1 Sport and Tournament Type

- **Selected sport:** Soccer
- **Category:** Team-based sport
- **Tournament format:** League-style tournament organized in rounds

Soccer was selected due to its widespread familiarity and well-defined structure involving teams, matches, results, standings, and strong public interest. Although the system is focused on soccer, the domain model is designed to be extensible to other team-based sports.

3. Stakeholders and Users

3.1 Stakeholders

The primary stakeholders involved in the system are:

- **Tournament organizers**, who define operational requirements
- **Course instructor**, who evaluates the architecture and implementation
- **The development team**, responsible for system design and implementation

3.2 User Roles and Expected Scale

The system supports the following user roles:

- **Administrator**
  - Manages tournaments, venues, teams, matches, results, and standings
  - Estimated users: 15–20
- **Coach / Team Manager**
  - Manages team information and player rosters
  - Estimated users: Up to 100
- **Referee / Match Official**
  - Records match events and submits official match results
  - Estimated users: Approximately 50
- **Spectator (Public User)**
  - Accesses tournament information through a public, read-only website
  - Estimated users: Up to 30,000–50,000 concurrent users during peak events

4. Functional Requirements

The following functional requirements are implemented in the distributed system:

- **FR-1: Team Enrollment**
  Teams can be registered for tournaments with basic team information.
- **FR-2: Player Management**
  Teams can register players with attributes such as name, position, and jersey number.
- **FR-3: Match Pairing and Scheduling**
  Matches are created, paired, scheduled, and assigned to venues and tournament rounds.
- **FR-4: Match Event Tracking**
  Referees can record match events such as goals, cards, and substitutions.
- **FR-5: Match Results and Reports**
  Final scores and official match reports are stored after each match.
- **FR-6: Standings and Rankings**
  Tournament standings are automatically calculated based on match results.
- **FR-7: Public Tournament Website**
  Spectators can view tournaments, matches, results, standings, and match reports via a public web interface.

5. Non-Functional Requirements

- **NFR-1: Scalability**
  Support large numbers of spectators and authenticated users during peak tournament periods.
- **NFR-2: Performance**
  Match events and results must be processed with low latency; public pages must load quickly.
- **NFR-3: Availability and Reliability**
  Confirmed match data must not be lost; public tournament information must remain accessible.

- **NFR-4: Maintainability and Modularity**
  The system must be modular and extensible.
- **NFR-5: Security**
  Only authorized users can modify tournament data.
- **NFR-6: Deployability**
  The system must support reproducible and low-risk deployments.
- **NFR-7: Cost Efficiency**
  Use open-source technologies and minimize unnecessary infrastructure usage.

6. Implemented Distributed Microservices Architecture

The system is implemented using a distributed microservices architecture, decomposed into independent services responsible for specific domain concerns.

6.1 Service Decomposition

The system consists of five core microservices:

1. Authentication and Authorization Service (Auth Service)

- Port: 8001
- Responsibilities: OAuth2 authentication, RBAC, token issuance and validation
- Database: auth_db

2. Tournament Management Service

- Port: 8002
- Responsibilities: Tournament lifecycle, sports and venues
- Database: tournament_db

3. Team and Player Management Service

- Port: 8003

- Responsibilities: Teams, players, squad management
- Database: team_db

4. Match Management Service

- Port: 8004
- Responsibilities: Match scheduling, match status, event tracking, reports
- Database: match_db

5. Results and Statistics Service

- Port: 8005
- Responsibilities: Standings calculation, statistics aggregation, top scorers
- Database: results_db

Each service follows the **database-per-service pattern**, ensuring domain data ownership and isolation.

6.2 Communication Patterns

Synchronous Communication

- RESTful HTTP APIs
- Token validation via Auth Service
- Direct client-to-service communication

Asynchronous Communication

- Redis Pub/Sub event-driven communication
- Example events:
    - match.completed
    - match.scheduled
    - standings.updated

    - tournament.created

6.3 Real-Time Communication

- WebSocket-based updates using Pusher
- Laravel Echo in frontend
- Live broadcast of match events within ~1 second

6.4 Frontend Applications

Admin Dashboard

- React + Vite
- Port: 3000
- Protected APIs (OAuth2)

Public View

- React + Vite
- Port: 3001
- Read-only public APIs

6.5 API Gateway Strategy

The current implementation does not include a dedicated API Gateway. Frontend applications communicate directly with backend microservices via REST APIs.

Authentication tokens issued by the Auth Service are validated independently by each service. While this simplifies development in an academic context, the architecture can be extended by introducing an API Gateway for centralized routing, rate limiting, and monitoring.

7. Technology Stack

Backend

- Laravel 11/12
- PHP 8.2+
- MySQL 8.0 (database per service)
- Redis 7
- Laravel Passport (OAuth2)
- Pusher (WebSocket)

Frontend

- React 18
- Tailwind CSS

Infrastructure

- Docker & Docker Compose
- Git

Task 2 – Architecture Characteristics

1. Scalability

Quantification

- 30,000–50,000 concurrent spectators
- < 2 seconds average response time under peak load

Architectural Support

The implemented microservices architecture enables independent horizontal scaling of each service container. Public-facing services (e.g., Match and Results services) can be replicated to handle high spectator traffic without scaling internal administrative services.

This ensures efficient resource allocation and prevents unnecessary scaling of low-traffic components.

2. Elasticity

Quantification

- Scale resources within 5 minutes
- Handle 8–10× sudden traffic bursts

Architectural Support

Docker-based containerization allows dynamic scaling of individual services based on demand. Services can be replicated or reduced independently, ensuring the system adapts quickly to varying tournament traffic patterns while maintaining performance stability.

3. Performance and Responsiveness

Importance

Referees and administrators require fast system feedback when recording match events and submitting results. In addition, spectators expect near real-time updates of scores and standings to ensure an engaging and reliable user experience.

Derived From Requirements

- FR-4: Match Event Tracking
- FR-5: Match Results and Reports
- FR-6: Standings and Rankings
- NFR-2: Performance

Quantification

- Match event submissions shall be processed in under 1 second.
- Final match result submissions shall be processed in under 1 second.
- Public pages (matches, results, standings) shall load in under 2 seconds under normal operating conditions.
- Updated standings shall be visible to spectators within 5 seconds after result submission.

The architecture supports these targets through service isolation, optimized database access per service, and event-driven updates.

4. Reliability

Quantification

- 99.5% availability during tournament operation
- Zero loss of confirmed match data

Architectural Support

Fault isolation ensures that failure in one service (e.g., Match Service) does not impact unrelated services. Each service exposes a standardized /health endpoint for container-level health checks.

Database-per-service isolation prevents cascading data corruption and ensures transactional integrity within each bounded context.

5. Deployability

Quantification

- Deployment time < 10 minutes
- Independent service deployment

Architectural Support

Docker Compose enables reproducible deployments across environments. Each service can be deployed or updated independently without requiring full system redeployment, reducing downtime and operational risk.

6. Modularity

Importance

The system must be maintainable and extendable, for example to support advanced statistics modules or additional sports in future iterations.

Architectural Support

The system is divided into bounded contexts:

- **Auth Service** → Identity & access control
- **Tournament Service** → Tournament configuration
- **Team Service** → Teams & players
- **Match Service** → Match lifecycle management
- **Results Service** → Standings & statistics

Each service:

- Owns its database
- Encapsulates its domain logic
- Supports independent deployment

This separation reduces coupling, increases cohesion, and improves long-term maintainability and extensibility.

7. Security

Architectural Support

- OAuth2 authentication using Laravel Passport
- Role-Based Access Control (RBAC) for Administrator, Coach, and Referee roles
- JWT validation at service boundaries
- Public APIs restricted to read-only operations
- Input validation and ORM-based query handling to prevent injection attacks

This layered security model protects both administrative operations and public data access.

8. Overall Cost Efficiency

Architectural Support

- Open-source technology stack
- Independent service scaling to optimize resource usage
- Container-based deployment for infrastructure efficiency

The architecture ensures cost control by scaling only necessary components and avoiding monolithic over-provisioning.

9. Real-Time Capabilities

Quantification

- Live match updates < 1 second
- Standings propagation < 5 seconds

Architectural Support

An event-driven architecture combined with WebSocket broadcasting ensures near real-time visibility of match events and standings updates.

This design supports high spectator engagement without overloading core transactional services.

10. Observability and Monitoring

Architectural Support

- /health endpoints available on all services
- Centralized Docker logging
- Standardized error response formats
- Service-level logging for debugging and auditing

These mechanisms improve operational transparency and simplify fault detection and diagnosis.

Conclusion

The implemented distributed microservices architecture successfully satisfies the defined functional and non-functional requirements. Through independent services, database-per-service isolation, event-driven communication, and containerized

deployment, the system demonstrates the practical application of modern software architecture principles in a production-style environment.

The architecture ensures scalability, modularity, reliability, security, performance, and real-time responsiveness, while maintaining cost efficiency through an open-source technology stack and efficient resource utilization.